Task 11: Hashing, Direction Sense & Coding-Decoding

28 Sept 2026 · @Snuggles

Data Structures & Algorithms — HashMap, HashSet, Frequency Counting + Direction Sense (Reasoning) + Coding-Decoding (Aptitude) · Task 11

Task 8 aur 9 mein sorting, Task 10 mein searching ho gayi. Ab agla step hai **Hashing** — jo bina sort kiye, lagbhag ek hi step (O(1)) mein bata deta hai ki element hai ya nahi. Yahi LeetCode Roadmap ka Day 1–4 (Arrays & Hashing) bhi hai, isliye Hardik ke problems seedha roadmap se hain. Akshara ke liye hashing basics + Kunal Kushwaha ke videos, aur saath mein Reasoning (Direction Sense) + Aptitude (Coding-Decoding).

1 Theory: Hashing (HashMap & HashSet)

Hashing ka matlab: har key ko ek **hash function** se ek number (bucket index) mein badlo, aur value seedha us bucket mein rakh do. Dhundte waqt bhi wahi function chalao aur seedha sahi bucket pe pahuncho — poora array scan nahi karna. Isliye insert, search aur delete **average O(1)** mein hote hain.

Analogy: Railway ka PNR number. Tum PNR daalte ho aur booking seedha khul jaati hai — system poori train ki list scroll nahi karta. PNR = key, booking = value, aur PNR se record tak pahunchne ka tareeka = hash function.

Collision kya hai? Maan lo class mein roll number ke last digit se bench milti hai — roll 23 → bench 3, roll 47 → bench 7. Ab roll 13 aur 23 dono bench 3 pe aa gaye — yahi **collision** hai. Java ek bucket ke andar kai items ek list mein rakh leta hai (chaining), aur Java 8+ mein bahut bhare hue bucket ko tree bana deta hai.

Kitna fast hai? — Searching Showdown (Task 10 vs Task 11)

Method	Search time	Sorted chahiye?	Extra space	Kab use karein
Linear Search	O(N)	Nahi	O(1)	Chhota data, ek-do baar search
Binary Search	O(log N)	Haan (sort = O(N log N))	O(1)	Data pehle se sorted ho
Hashing (HashSet / HashMap)	O(1) average	Nahi	O(N)	Baar-baar "hai ya nahi?" ya "kitni baar?"

Trade-off yaad rakho: hashing **speed ke badle memory** leta hai (extra O(N) space).

HashSet vs HashMap

	HashSet	HashMap
Kya store karta hai	Sirf unique elements	Key → Value pairs
Kis sawaal ka jawab	"Yeh element hai ya nahi?"	"Kitni baar aaya?" / "Kis index par tha?"
Duplicate daalo toh	Add hi nahi hota — add() false deta hai	Purani value overwrite — put() purani value return karta hai
Example problem	Contains Duplicate	Two Sum, frequency count

Java Cheat Sheet

```
import java.util.*;

HashSet<Integer> set = new HashSet<>();
set.add(5);                 // true  (naya element)
set.add(5);                 // false (duplicate — add hi nahi hua)
set.contains(5);            // true  → O(1) search!
set.remove(5);

HashMap<String, Integer> map = new HashMap<>();
map.put("apple", 1);
map.put("apple", map.getOrDefault("apple", 0) + 1);  // apple → 2
```

```
map.get("mango");           // null (key hi nahi hai)
map.containsKey("apple");   // true
for (Map.Entry<String, Integer> e : map.entrySet()) {
    System.out.println(e.getKey() + " = " + e.getValue());
}
```

Frequency Counting Template (is task ka sabse important pattern)

```
static HashMap<Integer, Integer> countFreq(int[] arr) {
    HashMap<Integer, Integer> freq = new HashMap<>();
    for (int x : arr) {
        freq.put(x, freq.getOrDefault(x, 0) + 1);
    }
    return freq;
}
```

Desi hashing: jab values ki range chhoti ho (jaise sirf 'a' se 'z'), toh HashMap ki jagah seedha `int[] count = new int[26]` aur `count[ch - 'a']++` likho. Array ka index hi key ban jaata hai — HashMap se bhi fast.

5 Golden Rules:

1. HashMap **order yaad nahi rakhta** — print karoge toh keys kisi bhi order mein aa sakti hain. Insertion order chahiye toh `LinkedHashMap`, sorted order chahiye toh `TreeMap`.
2. `map.get(key)` missing key pe `null` deta hai. Usko seedha `int` mein daaloge toh **NullPointerException** — `getOrDefault(key, 0)` use karo.
3. Map se nikli `Integer` values ko `==` se compare mat karo — `.equals()` use karo (Task 10 ka Case 9 yaad hai?).
4. Key unique hoti hai — same key pe dobara `put()` purani value ko overwrite kar deta hai.
5. For-each loop chalte-chalte map/set se `remove()` mat karo — `ConcurrentModificationException` aayega. `iterator.remove()` ya `removeIf()` use karo.

→ GeeksforGeeks: Hash Table Data Structure

→ GeeksforGeeks: HashMap in Java

→ GeeksforGeeks: HashSet in Java

→ Java Docs: HashMap class

2 DSA Practice Problems

Akshara ke liye

Searching → Hashing — pichle task (Array Search, Binary Search Basic) ka agla step. Problems se pehle yeh videos dekho.

Reference videos — Kunal Kushwaha (Java + DSA Bootcamp):

#	Video	Kya dekhna hai
1	Introduction to HashMap & HashTable in Java	Is task ka main video. "How HashMaps work" wala part zaroor dekhna; chaining / open addressing ka implementation abhi optional hai.
2	Strings and StringBuilder in Java	charAt(), toCharArray() aur characters ke saath kaam — Q3 aur Q4 ke liye zaroori.
3	Arrays & ArrayList in Java	Link ArrayList wale part se khulta hai — Collections ka basic; HashMap bhi isi family ka hissa hai.
4	Linear Search + Binary Search	Task 10 ka revision — compare karo ki hashing inse fast kyun hai.

Poori playlist: Kunal Kushwaha — Java + DSA + Interview Preparation Course

1. **Frequencies in a Limited Array** · GFG
   - Har number (1 se n tak) array mein kitni baar aaya, count karo. Pehle do loops wala O(N²) likho, phir ek count array se O(N).
   - *Hint:* `count[arr[i]]++` — array ka index hi "key" ban gaya. Yahi hashing ka desi version hai.
   - → GeeksforGeeks: Frequencies in a Limited Array
2. **Contains Duplicate** · LeetCode 217
   - Koi number do baar aaya? true / false return karo.
   - *Hint:* HashSet mein daalte jao — `add()` false de toh duplicate mil gaya. Bonus: sort karke adjacent elements check karo (Task 9 ki sorting) aur dono approaches ka time compare karo.
   - → leetcode.com/problems/contains-duplicate
3. **Valid Anagram** · LeetCode 242
   - Kya t, s ke letters ko rearrange karke bana hai? (listen → silent = true)

- *Hint:* `int[26]` count array — s ke har letter pe ++, t ke har letter pe --. End mein saare 0 hone chahiye.
- → leetcode.com/problems/valid-anagram

4. **First Unique Character in a String** · LeetCode 387
   - Pehla character jo sirf ek baar aata hai, uska index return karo; na ho toh -1.
   - *Hint:* Do pass — pehle frequency count karo, phir string dobara padho aur pehla count == 1 wala index return karo.
   - → leetcode.com/problems/first-unique-character-in-a-string
5. **Intersection of Two Arrays** · LeetCode 349
   - Dono arrays ke common elements return karo (har element sirf ek baar).
   - *Hint:* Pehle array ko HashSet mein daalo; doosre array ke har element pe `contains()`. Yeh Linear Search ka O(1) version hai!
   - → leetcode.com/problems/intersection-of-two-arrays
6. **Two Sum** · LeetCode 1
   - Do numbers jinka sum = target, unke index return karo.
   - *Hint:* Pehle brute force (do loops, O(N²)). Phir `HashMap<value, index>`: har x pe check karo ki (target − x) pehle dekha hai kya — one pass, O(N). Interview ka sabse famous question!
   - → leetcode.com/problems/two-sum

Hardik ke liye

LeetCode Roadmap Day 1–4 (Arrays & Hashing) — Medium focus, time complexity matter karegi.

Warm-up (roadmap ke 3 Easy, 10–15 min each): Contains Duplicate (217), Valid Anagram (242), Two Sum (1) — seedha optimal O(N) solution likho.

1. **Product of Array Except Self** · LeetCode 238 · Roadmap #2
   - Har index pe baaki sab elements ka product — division ke bina, O(N) mein.
   - *Hint:* Left se prefix product, right se suffix product. Output array ke alawa O(1) extra space mein karke dikhao. Yeh "prefix" wali soch Q6 mein kaam aayegi.
   - → leetcode.com/problems/product-of-array-except-self
2. **Valid Sudoku** · LeetCode 36 · Roadmap #3
   - *Hint:* Har row, column aur 3×3 box ke liye ek HashSet (total 27). Box number = (r / 3) * 3 + c / 3. Ya ek hi `HashSet<String>` mein "5 in row 0", "5 in col 3", "5 in box 1" jaisi strings daalo.
   - → leetcode.com/problems/valid-sudoku
3. **Group Anagrams** · LeetCode 49 · Roadmap #6

- *Hint:* `HashMap<String, List<String>>`. Key = sorted word (O(K log K)) ya 26-count signature (O(K)). Tricky Code ka Case 7 dekho — `char[]` ko seedha key mat banana!
- → leetcode.com/problems/group-anagrams

4. **Top K Frequent Elements** · LeetCode 347 · Roadmap #7
   - *Hint:* Frequency map + Bucket Sort (bucket index = frequency) → O(N). Task 9 ke Kth Largest wala Min-Heap approach O(N log K) bhi likho aur dono compare karo.
   - → leetcode.com/problems/top-k-frequent-elements
5. **Longest Consecutive Sequence** · LeetCode 128 · Roadmap #8
   - *Hint:* O(N) chahiye, sort (O(N log N)) nahi. Sab kuch HashSet mein daalo; count sirf wahan se shuru karo jahan (x − 1) set mein nahi hai — wahi sequence ki shuruaat hai.
   - → leetcode.com/problems/longest-consecutive-sequence
6. **Subarray Sum Equals K** · LeetCode 560 · Roadmap #9
   - *Hint:* Prefix sum + `HashMap<prefixSum, count>`, shuru mein `map.put(0, 1)`. Array mein negative numbers ho sakte hain — isliye sliding window kyun fail hoti hai, README.md mein 2 lines mein likho.
   - → leetcode.com/problems/subarray-sum-equals-k

Roadmap tracker update: LeetCode_150_Roadmap.xlsx mein Day 1–4 ke saare 9 rows (#1 se #9) tick karo. Pichle tasks mein ho chuke problems bhi tick kar do: Binary Search (#31), Find First and Last Position (#32), Find the Duplicate Number (#45), Kth Largest Element in an Array (#69) aur Merge Intervals (#133).

3 Reasoning: Direction Sense (Akshara ke liye)

Direction Sense mein koi insaan kai turns leta hai, aur tumhe batana hota hai ki woh akhir mein **kahan hai** (start se kitni door, kis direction mein) ya **kis taraf face kar raha hai**. Yeh placements, SSC, bank exams aur CAT — sab mein aata hai. Aur yeh bilkul code ke dry run jaisa hai: har step pe variables (x, y, facing) update karte jao.



Sochne ka tarika

1. **Hamesha kaagaz pe compass banao** — N upar, E right. Dimaag mein ghumaoge toh galti pakki.
2. **Right turn = clockwise 90°, Left turn = anticlockwise 90°** — tumhare chehre ke hisaab se, naksha ke hisaab se nahi. North face karke right = East, lekin South face karke right = West!
3. **Coordinate trick (x, y):** Start = (0, 0). East → x badhao, West → x ghatao, North → y badhao, South → y ghatao. Har step ke baad (x, y) likho — bilkul variables ka dry run.
4. **Final distance = √(x² + y²)** (Pythagoras). Common triples yaad rakho: 3-4-5, 6-8-10, 5-12-13, 8-15-17.
5. **Direction ka naam:** x aur y ke sign dekho — (+, +) = North-East, (−, +) = North-West, (−, −) = South-West, (+, −) = South-East. x = 0 ho toh seedha North ya South.
6. **Shadow rule:** Subah suraj East mein hota hai → parchhai West ki taraf girti hai. Shaam ko suraj West mein → parchhai East ki taraf.
7. **Degree turns:** Bearing se socho — N = 0°, E = 90°, S = 180°, W = 270°. Clockwise = jodo (+), anticlockwise = ghatao (−), phir 360 ka mod lo. 45° = aadha right angle.
8. **"Kahan hai" aur "kis taraf face kar raha hai" do alag sawaal hain** — question dhyaan se padho.

Abhi face kar rahe ho	Right turn ke baad	Left turn ke baad	U-turn (180°) ke baad
North	East	West	South
East	South	North	West
South	West	East	North
West	North	South	East

15 Hard Practice Questions

Har question ke liye DirectionSense.md mein answer + steps (coordinates ya chhota diagram) likhna hai — sirf answer nahi chalega.

LEVEL 1 — MULTI-TURN WALKS

Q1. Riya starts from her home and walks 10 km towards North. She turns right and walks 6 km, turns right again and walks 4 km, then turns left and walks 2 km, and finally turns left and walks 9 km. (i) How far is she from her home, and in which direction? (ii) Which direction is she facing now?

Q2. A robot starts at point O facing East. It moves 1 m, turns left, moves 2 m, turns left, moves 3 m, and keeps following this pattern (turn left, then move 1 m more than the previous move). After its 8th move, how far is it from O, in which direction, and which way is it facing? *(Hint: loop ki tarah socho — for i = 1 to 8.)*

Q3. Starting from point P, Karan walks 12 m West. He turns 90° anticlockwise and walks 5 m, turns 90° anticlockwise again and walks 24 m, then turns 90° clockwise and walks 5 m, turns 90° clockwise again and walks 12 m, and finally turns 90° clockwise and walks 7 m. How far and in which direction is he from P?

Q4. Aman and Bhavna start from the same point. Aman walks 8 km North, turns right and walks 3 km, then turns right again and walks 2 km. Bhavna walks 5 km South, turns left and walks 9 km, then turns left again and walks 3 km. How far apart are they now, and in which direction is Bhavna from Aman?

LEVEL 2 — DEGREES, SHADOWS & CLOCKS

Q5. Neeraj is facing North-East. He turns 135° clockwise, then 270° anticlockwise, and then 45° clockwise. Which direction is he facing now?

Q6. One evening, just before sunset, Sameer and Tanya were standing face to face in a park. Tanya's shadow fell exactly to Sameer's left. Which direction was Sameer facing?

Q7. A clock is kept on a table so that at 3:00 PM its minute hand points towards North-East. In which direction will the hour hand point at 7:30 PM?

Q8. Pooja walks 20 m towards East, turns left and walks 20 m. She then turns 135° clockwise and walks $20\sqrt{2}$ m. How far and in which direction is she from her starting point?

LEVEL 3 — RELATIVE POSITIONS

Q9. Point Q is 6 km East of point P. R is 8 km North of Q. S is 6 km West of R. T is 4 km South of S. U is 3 km East of T. (i) In which direction is U from P, and how far? (ii) What is the distance between U and Q?

Q10. Six friends A, B, C, D, E and F stand in a ground. A is 10 m West of B. C is 10 m South of B. D is 10 m East of C. E stands exactly midway between A and D. F is 5 m East of E. (i) In which direction is F from C, and how far? (ii) Who stands exactly midway between B and C?

Q11. Village Q is North of village P. Village R is East of Q. Village S is West of P. Village T is South of S. No distances are given. In which direction is R from T? Explain why the answer does not depend on the distances.

LEVEL 4 — CODED DIRECTIONS

Code: P $ Q → P is 5 m North of Q | P # Q → P is 5 m East of Q | P @ Q → P is 5 m South of Q | P % Q → P is 5 m West of Q

Q12. If A $ B # C @ D, in which direction is A from D, and how far?

Q13. Which expression shows that X is to the North-East of Z? (a) X $ Y % Z (b) X # Y $ Z (c) X @ Y # Z (d) X % Y @ Z

LEVEL 5 — ROTATED COMPASS

Q14. If North-East becomes West, and all other directions change in the same manner, what will South become?

Q15. In a strange town, directions are renamed so that North is called South-East, and all other directions are renamed in the same manner. A man walks 10 km towards the direction the town calls "West", then turns left and walks 10 km. In real directions, how far and in which direction is he from his starting point?

Aur practice chahiye?

→ IndiaBix: Direction Sense Test (practice questions with explanations)

→ PlacementPreparation: Direction Sense Concepts

4 Aptitude: Coding-Decoding (Akshara ke liye)

Coding-Decoding mein ek word ya number ko kisi **rule** se code kiya jaata hai — tumhe woh rule pakadna hai aur naye word pe lagana hai. TCS NQT, Infosys, Wipro, bank aur SSC exams sab mein yeh aata hai. Aur yeh literally programming hai: input → rule (function) → output.

Must-know tricks

1. **Alphabet positions:** A = 1 … Z = 26. Shortcut **EJOTY** = 5, 10, 15, 20, 25 — beech ke letters inhi se gin lo.
2. **Opposite letters:** har jodi ka sum 27 hota hai — A↔Z, B↔Y, C↔X, D↔W, E↔V, F↔U, G↔T, H↔S, I↔R, J↔Q, K↔P, L↔O, M↔N.
3. **Wrap-around:** Z ke baad phir A aata hai (Y + 3 = B, A − 2 = Y). Java mein yahi hai `(ch - 'A' + k) % 26 + 'A'`.
4. **Rule pehle 2–3 letters pe check karo,** phir poore word pe lagao. Rule har letter pe fit hona chahiye.
5. **Sentence coding mein common words dhundo:** jo word do sentences mein common hai, uska code bhi dono mein common hoga — bilkul do HashSets ka intersection (Akshara ka Q5 yaad hai?).

Type	Rule	Example
Fixed shift	Har letter +2	CAT → ECV
Increasing shift	+1, +2, +3 …	BRAIN → CTDMS
Reverse + shift	Word ulta karo, phir +1	GARDEN → OFESBH
Opposite letters	Jodi ka sum 27	WATER → DZGVI
Number coding	Letter → position ya uska square	ACE → 1925
Sentence coding	Common words se codes nikalo	"sky is blue" → "ta na pa"
Conditional coding	Table + if-else rules	Neeche Level 4 dekho

15 Medium Practice Questions

Har question ke liye CodingDecoding.md mein answer + jo rule tumne pakda, woh likhna hai.

LEVEL 1 — LETTER CODING

Q1. In a certain code, BRAIN is written as CTDMS. How is SMART written in that code?

Q2. If GARDEN is written as OFESBH, how is FLOWER written in that code?

Q3. In a certain code, WATER is written as DZGVI. How is SMILE written in that code?

Q4. In a certain code, MOTHER is written as NMUFFP. How is FATHER written in that code?

Q5. If COMPUTER is written as PMOCRETU, how is KEYBOARD written in that code?

LEVEL 2 — NUMBER CODING

Q6. If PAINT is coded as 74128 and EXCEL is coded as 93596, how is ACCEPT coded?

Q7. In a certain code, ACE is written as 1925 and BAD is written as 4116. How is FIG written?

Q8. If CAT = 72, DOG = 78 and FISH = 168, then LION = ?

LEVEL 3 — SENTENCE, SUBSTITUTION & DECODING

Q9. In a certain code language, 'sky is blue' is written as 'ta na pa', 'blue and green' is written as 'ka ta ma', and 'green is fresh' is written as 'na ma ra'. What is the code for 'sky'?

Q10. If 'pen' is called 'pencil', 'pencil' is called 'eraser', 'eraser' is called 'sharpener', and 'sharpener' is called 'scale', what will a student use to rub out a pencil mark?

Q11. In a code, every letter is replaced by the letter 3 places after it in the alphabet (X → A, Y → B, Z → C). Decode this message: **KDVK PDS**

LEVEL 4 — CONDITIONAL CODING (Q12 - Q15)

Letters are coded using this table:

Letter	A	B	D	E	G	K	M	P	R	T
Code	5	@	3	#	9	1	%	7	$	2

Conditions:

1. If the first letter is a vowel and the last letter is a consonant, the codes of the first and the last letters are interchanged.
2. If both the first and the last letters are consonants, both are coded as `*`.
3. If the first letter is a consonant and the last letter is a vowel, the last letter is coded as the code of the first letter.

If no condition applies, code every letter as per the table.

Q12. EGPKMT **Q13.** MBRADK **Q14.** TGAKPE **Q15.** AKDTPE

Aur practice chahiye?

→ FACE Prep: 7 Types of Coding-Decoding Questions (worked examples)

→ PlacementPreparation: Coding and Decoding Questions & Answers

→ Sawaal: Coding and Decoding practice with explanations

5 Tricky Code Analysis (10 Questions)

Dono ke liye. In codes ko dry run karo. Output predict karo, ya agar compilation / runtime error hai toh reason batao.

Case 1 — put() Return Value & Duplicate Keys

```
import java.util.HashMap;

public class Test1 {
    public static void main(String[] args) {
        HashMap<String, Integer> map = new HashMap<>();
        map.put("apple", 10);
        map.put("banana", 20);
        System.out.println(map.put("apple", 30));
```

```
        System.out.println(map.size());
        System.out.println(map.get("apple"));
    }
}
```

Case 2 — get() on a Missing Key

```
import java.util.HashMap;

public class Test2 {
    public static void main(String[] args) {
        HashMap<Character, Integer> freq = new HashMap<>();
        freq.put('a', 2);
        System.out.println(freq.get('b'));
        int count = freq.get('b');
        System.out.println(count);
    }
}
```

Case 3 — HashSet add() Return Value

```
import java.util.HashSet;

public class Test3 {
    public static void main(String[] args) {
        HashSet<Integer> set = new HashSet<>();
        System.out.println(set.add(5));
        System.out.println(set.add(5));
        set.add(7);
        System.out.println(set.size() + " " + set.contains(7));
    }
}
```

Case 4 — Removing While Iterating

```
import java.util.HashMap;

public class Test4 {
    public static void main(String[] args) {
        HashMap<Integer, String> map = new HashMap<>();
        map.put(1, "one");
        map.put(2, "two");
        map.put(3, "three");
        for (Integer key : map.keySet()) {
            if (key == 2) {
                map.remove(key);
```

```
            }
        }
        System.out.println(map);
    }
}
```

Case 5 — HashMap vs LinkedHashMap vs TreeMap Order

```
import java.util.*;

public class Test5 {
    public static void main(String[] args) {
        Map<Integer, String> hm = new HashMap<>();
        Map<Integer, String> lhm = new LinkedHashMap<>();
        Map<Integer, String> tm = new TreeMap<>();
        for (int k : new int[]{30, 10, 20}) {
            hm.put(k, "x");
            lhm.put(k, "x");
            tm.put(k, "x");
        }
        System.out.println(hm.keySet());
        System.out.println(lhm.keySet());
        System.out.println(tm.keySet());
    }
}
```

Case 6 — Comparing Values Taken from Maps

```
import java.util.HashMap;

public class Test6 {
    public static void main(String[] args) {
        HashMap<String, Integer> a = new HashMap<>();
        HashMap<String, Integer> b = new HashMap<>();
        a.put("x", 100);  b.put("x", 100);
        a.put("y", 500);  b.put("y", 500);
        System.out.println(a.get("x") == b.get("x"));
        System.out.println(a.get("y") == b.get("y"));
        System.out.println(a.get("y").equals(b.get("y")));
        System.out.println(a.equals(b));
    }
}
```

Case 7 — Arrays as HashSet Keys

```java
import java.util.Arrays;
import java.util.HashSet;

public class Test7 {
    public static void main(String[] args) {
        HashSet<int[]> set1 = new HashSet<>();
        set1.add(new int[]{1, 2});
        System.out.println(set1.contains(new int[]{1, 2}));

        HashSet<String> set2 = new HashSet<>();
        set2.add(Arrays.toString(new int[]{1, 2}));
        System.out.println(set2.contains(Arrays.toString(new int[]{1, 2})));
    }
}
```

Case 8 — Capital Letters in a Count Array

```java
public class Test8 {
    public static void main(String[] args) {
        String s = "Java";
        int[] count = new int[26];
        for (char ch : s.toCharArray()) {
            count[ch - 'a']++;
        }
        System.out.println(count[0]);
    }
}
```

Case 9 — Printing a Frequency Map

```java
import java.util.HashMap;

public class Test9 {
    public static void main(String[] args) {
        int[] arr = {3, 1, 3, 2, 3, 1};
        HashMap<Integer, Integer> freq = new HashMap<>();
        for (int x : arr) {
            freq.put(x, freq.getOrDefault(x, 0) + 1);
        }
        System.out.println(freq);
        System.out.println(freq.getOrDefault(5, -1));
        System.out.println(freq.containsValue(2));
    }
}
```

Case 10 — null Keys and null Values

```java
import java.util.HashMap;

public class Test10 {
    public static void main(String[] args) {
        HashMap<String, String> map = new HashMap<>();
        map.put(null, "A");
        map.put("k", null);
        map.put(null, "B");
        System.out.println(map.size());
        System.out.println(map.get(null));
        System.out.println(map.get("k") + " " + map.containsKey("k"));
        System.out.println(map.get("z") + " " + map.containsKey("z"));
    }
}
```

6 Code Modification Challenge

Neeche ek basic frequency counter diya gaya hai jo do nested loops se har element ki count nikalta hai. Isme do problem hain: yeh O(N²) hai, aur repeated elements ko baar-baar print karta hai (4 teen baar print hoga).

FrequencyCounter.java — Starting Point

```java
public class FrequencyCounter {
    public static void main(String[] args) {
        int[] arr = {4, 7, 4, 2, 9, 7, 4, 1, 9, 3};
        for (int i = 0; i < arr.length; i++) {
            int count = 0;
            for (int j = 0; j < arr.length; j++) {
                if (arr[i] == arr[j]) {
                    count++;
                }
            }
            System.out.println(arr[i] + " -> " + count);
        }
    }
}
```

Tumhara Task — Is code ko modify karo taaki yeh:

1. Nested loops hata kar `HashMap<Integer, Integer>` se **O(N)** mein frequency nikale, aur har element **sirf ek baar** print ho.
2. Output usi order mein aaye jis order mein element array mein pehli baar aaya tha (4, 7, 2, 9, 1, 3). *(Hint: `LinkedHashMap`.)*

3. Sabse zyada baar aane wala element (mode) aur uski count print kare. Tie ho toh chhota number.
4. Pehla **non-repeating** element print kare. Koi na ho toh "None" print kare.
5. Dono versions mein loop kitni baar chala (steps), count karke print kare — nested loops vs HashMap, N = 10 ke liye compare karo.

Bonus (optional) — CodeDecoder.java: ek method `shift(String word, int k)` likho jo har capital letter ko k position aage kar de, wrap-around ke saath (Z + 1 = A). Negative k se decode bhi ho jaana chahiye. Isse Aptitude ka Q11 verify karo; phir ek chhota change karke (i-th letter ko i + 1 shift) Q1 bhi verify karo.

7 Submission & Deployment

Dono students ko apna code task_11/ folder mein push karke GitHub link submit karni hai.

```
task_11/
├── Akshara/
|   ├── FrequencyCount.java
|   ├── ContainsDuplicate.java
|   ├── ValidAnagram.java
|   ├── FirstUniqueChar.java
|   ├── IntersectionOfArrays.java
|   ├── TwoSum.java
|   ├── DirectionSense.md    (15 answers + har ek ke coordinates / diagram)
|   └── CodingDecoding.md    (15 answers + har ek ka rule)
├── Hardik/
|   ├── ProductExceptSelf.java
|   ├── ValidSudoku.java
|   ├── GroupAnagrams.java
|   ├── TopKFrequent.java
|   ├── LongestConsecutive.java
|   ├── SubarraySumK.java
|   └── README.md            (har problem ki time/space complexity + LC 560
wala answer)
├── TrickyCodeAnalysis.md    (10 cases ke answers + reasoning)
├── FrequencyCounter.java    (modified — HashMap + order + mode + first
unique + step count)
└── CodeDecoder.java         (bonus — optional)
```